

Blocco 10 - Window function

Calcolare confronti mantenendo il dettaglio

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

A volte serve confrontare ogni riga con il suo gruppo senza perdere il dettaglio: ranking dei clienti, ricavo cumulato mensile, differenza rispetto al mese precedente. Una normale aggregazione ridurrebbe troppe righe.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Aggregare e perdere dettaglio

Situazione concreta

Per classificare paesi o clienti si può aggregare, ma spesso vogliamo mantenere ogni riga e aggiungere un confronto con il gruppo.

```
SELECT country, sum(revenue)
FROM sales
GROUP BY country;
```

Limiti della rappresentazione iniziale

- ▶ perde righe di dettaglio;
- ▶ non mostra ranking dentro gruppi;
- ▶ non confronta con periodo precedente;
- ▶ non calcola cumulati mantenendo il dettaglio.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Principio

Le window function calcolano ranking, cumulati e confronti guardando un insieme di righe correlate alla riga corrente.

Esempio di riorganizzazione

```
rank() OVER (  
  PARTITION BY month  
  ORDER BY revenue DESC  
) AS country_rank
```

- ① costruire la relazione base;
- ② scegliere la partizione;
- ③ scegliere l'ordinamento;
- ④ scegliere funzione e frame;
- ⑤ controllare pari merito e righe finali.

Analogia: classifica durante una gara

Analogia o caso esplicativo

Ogni atleta mantiene la propria scheda, ma accanto compare la posizione nella classifica della categoria. La riga non scompare: viene arricchita da un calcolo fatto guardando le altre righe del gruppo.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

- ▶ OVER;
- ▶ PARTITION BY;
- ▶ ORDER BY;
- ▶ frame di finestra;
- ▶ rank, lag, sum over.

- ▶ **Window function**: funzione calcolata su un insieme di righe correlate alla riga corrente.
- ▶ `PARTITION BY`: divide il dataset in gruppi indipendenti.
- ▶ `ORDER BY`: definisce sequenza dentro la finestra.
- ▶ **Frame**: sottoinsieme ordinato usato da alcune funzioni.
- ▶ `LAG/LEAD`: accesso a righe precedenti o successive.

Come riconoscere una buona soluzione

- ▶ la relazione base ha la granularità giusta;
- ▶ partizione e ordinamento sono coerenti con la domanda;
- ▶ il numero di righe finali è quello atteso;
- ▶ i pari merito sono trattati intenzionalmente.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Domanda: “Per ogni mese, classificare i paesi per ricavo, mantenendo una riga per mese e paese.”

Esempio: ragionamento

- ▶ prima serve aggregare il ricavo a mese-paese;
- ▶ poi la finestra calcola il ranking dentro ogni mese;
- ▶ la riga mese-paese resta nel risultato;
- ▶ ordinamento per ricavo decrescente e paese come tie-breaker.

Esempio completo: ranking mensile

```
WITH country_month AS (  
    SELECT date_trunc('month', r.order_date)::date AS month,  
           c.country,  
           round(sum(r.gross_revenue), 2) AS revenue  
    FROM order_revenue r  
    JOIN customers c ON c.customer_id = r.customer_id  
    WHERE r.status NOT IN ('cancelled', 'refunded')  
    GROUP BY month, c.country  
)  
SELECT month, country, revenue,  
       rank() OVER (PARTITION BY month ORDER BY revenue DESC, country) AS country_rank  
FROM country_month  
ORDER BY month, country_rank, country;
```

Errori frequenti da prevenire

- ▶ usare window function quando serve prima aggregare;
- ▶ dimenticare PARTITION BY;
- ▶ ordinare con colonne non deterministiche in caso di pari merito;
- ▶ confondere rank, dense_rank e row_number;
- ▶ non controllare se il frame include righe future o solo precedenti.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

Il management vuole leggere classifiche e trend: ranking dei paesi per mese, ricavo cumulato, confronto con mese precedente e quota percentuale del totale.

Esercitazione: specifica corretta

- ▶ input: aggregati di vendita validi;
- ▶ output: query con window function e colonne di confronto;
- ▶ vincolo: mantenere il dettaglio richiesto, non collassare righe inutilmente;
- ▶ vincolo: ogni finestra deve dichiarare partizione e ordinamento quando rilevanti.

Esercitazione: definizione della soluzione

- ① costruire CTE base con ricavo mensile;
- ② applicare ranking per partizione mensile;
- ③ usare `sum(...)` `OVER` per cumulati;
- ④ usare `lag` per confronto col periodo precedente.

Implementazione completa: cumulato per canale (1/2)

```
SET search_path TO training;

WITH country_month AS (
    SELECT date_trunc('month', r.order_date)::date AS month,
           c.country,
           round(sum(r.gross_revenue), 2) AS revenue
    FROM order_revenue r
    JOIN customers c ON c.customer_id = r.customer_id
    WHERE r.status NOT IN ('cancelled', 'refunded')
    GROUP BY month, c.country
)
SELECT month, country, revenue,
       rank() OVER (PARTITION BY month ORDER BY revenue DESC, country) AS country_rank,
       round(100 * revenue / sum(revenue) OVER (PARTITION BY month), 2) AS pct_month
FROM country_month
ORDER BY month, country_rank, country;
```

Implementazione completa: cumulato per canale (2/2)

```
WITH monthly_channel AS (  
    SELECT date_trunc('month', order_date)::date AS month,  
           channel,  
           round(sum(gross_revenue), 2) AS revenue  
    FROM order_revenue  
    WHERE status NOT IN ('cancelled', 'refunded')  
    GROUP BY month, channel  
)  
SELECT month, channel, revenue,  
       sum(revenue) OVER (PARTITION BY channel ORDER BY month  
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_revenue,  
       lag(revenue) OVER (PARTITION BY channel ORDER BY month) AS previous_month_revenue  
FROM monthly_channel  
ORDER BY channel, month;
```

Esercitazione: organizzazione dei 90 minuti

- ▶ 15 min confronto tra GROUP BY e finestra;
- ▶ 25 min ranking;
- ▶ 25 min cumulati e percentuali;
- ▶ 15 min LAG/LEAD;
- ▶ 10 min pari merito e frame.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ una window function arricchisce righe, non le raggruppa;
- ▶ partizione e ordinamento sono parte della semantica, non dettagli opzionali;
- ▶ prima si costruisce la relazione base, poi si applica la finestra.